

SENATI

Actividad #11

Programando nuestras API

Alumno:

Saavedra Donayre Alonso

Profesor:

Giancarlos Barboza Nieto

Curso:

Programación

Primero creamos las apis depende de donde la usemos, es decir nuestros apartados en nuestra pagina.

Api_alumnos.php

```
1  <?php
2  header("Content-Type: application/json; charset=UTF-8");
3
4  // ===== CONEXIÓN =====
5  $host = "localhost";
6  $db = "MATRICULA";
7  $user = "root";
8  $pass = "";
9
10 try {
11     $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
12     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
13 } catch (PDOException $e) {
14     echo json_encode(["mensaje" => "Error de conexión"]);
15     exit;
16 }
17
18 // ===== VARIABLES =====
19 $metodo = $_SERVER['REQUEST_METHOD'];
20 $datosJSON = json_decode(file_get_contents("php://input"));
21
22 switch ($metodo) {
23
24     // ===== GET =====
25     case 'GET':
26         try {
27
28             if (isset($_GET['id'])) {
29
30                 $sql = "SELECT * FROM ALUMNO WHERE ID_ALUMNO = :id";
31                 $stmt = $pdo->prepare($sql);
32                 $stmt->execute(['id' => $_GET['id']]);
33
34                 $alumno = $stmt->fetch(PDO::FETCH_ASSOC);
35
36                 echo json_encode($alumno ?: []);
37
38             } else {
39
40                 $sql = "SELECT * FROM ALUMNO";
41                 $stmt = $pdo->prepare($sql);
42                 $stmt->execute();
43
44                 $alumnos = $stmt->fetchAll(PDO::FETCH_ASSOC);
45                 echo json_encode($alumnos);
46             }
47
48         } catch (Exception $e) {
49             http_response_code(500);
50             echo json_encode(["mensaje" => "Error al obtener datos"]);
51         }
52     }
53 }
```

```

51     }
52     break;
53
54
55     // ===== POST =====
56     case 'POST':
57         try {
58
59             $sql = "INSERT INTO ALUMNO
60                 (DNI_ALUMNO, NOMBRES, APELLIDOS, FECHA_NACIMIENTO, EDAD, GENERO, DIRECCION, CELULAR,
61                 CORREO, NOMBRE_APODERADO, CELULAR_APODERADO, USERNAME, PASSWORD_HASH, estado)
62                 VALUES
63                 (:dni, :nombres, :apellidos, :fecha_nac, :edad, :genero, :direccion, :celular,
64                 :correo, :apoderado, :cel_apoderado, :username, :password, :estado)";
65
66             $stmt = $pdo->prepare($sql);
67
68             $stmt->execute([
69                 'dni' => $datosJSON->dni,
70                 'nombres' => $datosJSON->nombres,
71                 'apellidos' => $datosJSON->apellidos,
72                 'fecha_nac' => $datosJSON->fecha_nac,
73                 'edad' => $datosJSON->edad,
74                 'genero' => $datosJSON->genero,
75                 'direccion' => $datosJSON->direccion,
76                 'celular' => $datosJSON->celular,
77                 'correo' => $datosJSON->correo,
78                 'apoderado' => $datosJSON->apoderado,
79                 'cel_apoderado' => $datosJSON->cel_apoderado,
80                 'username' => $datosJSON->username,
81                 'password' => password_hash($datosJSON->password, PASSWORD_DEFAULT),
82                 'estado' => $datosJSON->estado
83             ]);
84
85             echo json_encode(["mensaje" => "Alumno creado"]);
86
87         } catch (Exception $e) {
88             http_response_code(500);
89             echo json_encode([
90                 "mensaje" => "Error al crear alumno",
91                 "error" => $e->getMessage()
92             ]);
93         }
94         break;
95
96
97     // ===== PUT =====
98     case 'PUT':
99         try {
100
101             $sql = "UPDATE ALUMNO SET
102                 DNI_ALUMNO=:dni,
103                 NOMBRES=:nombres,
104                 APELLIDOS=:apellidos,
105                 FECHA_NACIMIENTO=:fecha_nac,
106                 EDAD=:edad,
107                 GENERO=:genero,
108                 DIRECCION=:direccion,
109                 CELULAR=:celular,
110                 CORREO=:correo,
111                 NOMBRE_APODERADO=:apoderado,
112                 CELULAR_APODERADO=:cel_apoderado,
113                 USERNAME=:username,
114                 estado=:estado
115                 WHERE ID_ALUMNO=:id";
116
117             $stmt = $pdo->prepare($sql);
118

```

```

119         $stmt->execute([
120             'dni' => $datosJSON->dni,
121             'nombres' => $datosJSON->nombres,
122             'apellidos' => $datosJSON->apellidos,
123             'fecha_nac' => $datosJSON->fecha_nac,
124             'edad' => $datosJSON->edad,
125             'genero' => $datosJSON->genero,
126             'direccion' => $datosJSON->direccion,
127             'celular' => $datosJSON->celular,
128             'correo' => $datosJSON->correo,
129             'apoderado' => $datosJSON->apoderado,
130             'cel_apoderado' => $datosJSON->cel_apoderado,
131             'username' => $datosJSON->username,
132             'estado' => $datosJSON->estado,
133             'id' => $datosJSON->id_alumno
134         ]);
135
136         if (!empty($datosJSON->password)) {
137             $sqlPass = "UPDATE ALUMNO SET PASSWORD_HASH=:password WHERE ID_ALUMNO=:id";
138             $stmtPass = $pdo->prepare($sqlPass);
139             $stmtPass->execute([
140                 'password' => password_hash($datosJSON->password, PASSWORD_DEFAULT),
141                 'id' => $datosJSON->id_alumno
142             ]);
143         }
144
145         echo json_encode(["mensaje" => "Alumno actualizado"]);
146
147     } catch (Exception $e) {
148         http_response_code(500);
149         echo json_encode(["mensaje" => "Error al actualizar alumno"]);
150     }
151     break;
152
153     // ===== DELETE =====
154     case 'DELETE':
155         try {
156
157             $sql = "DELETE FROM ALUMNO WHERE ID_ALUMNO = :id";
158             $stmt = $pdo->prepare($sql);
159
160             $stmt->execute([
161                 'id' => $datosJSON->id_alumno
162             ]);
163
164             echo json_encode(["mensaje" => "Alumno eliminado"]);
165
166         } catch (Exception $e) {
167             http_response_code(500);
168             echo json_encode(["mensaje" => "Error al eliminar alumno"]);
169         }
170     }
171     break;
172
173     default:
174         http_response_code(405);
175         echo json_encode(["mensaje" => "Método no permitido"]);
176         break;
177     }
178 }
179 >>

```

POSTMAN -- Api_alumnos.php

Método Get

The screenshot shows a Postman interface with a GET request to `http://localhost/ProbandoApis/php/api_alumnos.php`. The response status is **200 OK** with a response time of 21 ms and a size of 1.47 KB. The response body is a JSON array of 4 student records. The first two records are fully visible in the image.

```
1  
2  
3  "ID_ALUMNO": 2,  
4  "DNI_ALUMNO": "73648291",  
5  "NOMBRES": "Camila Fernanda",  
6  "APELLIDOS": "Torres Quispe",  
7  "FECHA_NACIMIENTO": "2006-11-28",  
8  "EDAD": 18,  
9  "GENERO": "F",  
10 "DIRECCION": "Jr. Puno 514",  
11 "CELULAR": "976543218",  
12 "CORREO": "camila@gmail.com",  
13 "NOMBRE_APODERADO": "Rosa Quispe",  
14 "CELULAR_APODERADO": "934567812",  
15 "USERNAME": "ctorres",  
16 "PASSWORD_HASH": "$2y$10$examplehash2",  
17 "FECHA_REGISTRO": "2026-04-19 16:22:24",  
18 "estado": "activo"  
19 },  
20 {  
21   "ID_ALUMNO": 3,  
22   "DNI_ALUMNO": "71234568",  
23   "NOMBRES": "Diego Sebastian",  
24   "APELLIDOS": "Mendoza Cruz",  
25   "FECHA_NACIMIENTO": "2005-08-09",  
26   "EDAD": 19,  
27   "GENERO": "M",  
28   "DIRECCION": "SJL Mz B Lt 12",  
29   "CELULAR": "965432187",  
30   "CORREO": "diego@gmail.com",  
31   "NOMBRE_APODERADO": "Carlos Mendoza",  
32   "CELULAR_APODERADO": "945678123",  
33   "USERNAME": "dmendoza",  
34   "PASSWORD_HASH": "$2y$10$examplehash3",  
35   "FECHA_REGISTRO": "2026-04-19 16:22:24",  
36   "estado": "activo"  
37 },  
38 {  
39   "ID_ALUMNO": 4,  
40   "DNI_ALUMNO": "70192834",
```

Método Post

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost/ProbandoApis/php/api_alumnos.php`
- Body (raw JSON):**

```
1 {
2   "dni": "78945612",
3   "nombres": "Luis Alberto",
4   "apellidos": "Rojas Diaz",
5   "fecha_nac": "2005-06-15",
6   "edad": 19,
7   "genero": "M",
8   "direccion": "Los Olivos",
9   "celular": "987654321",
10  "correo": "luis@test.com",
11  "apoderado": "Maria Diaz",
12  "cel_apoderado": "912345678",
13  "username": "lrojas",
14  "password": "1234",
15  "estado": "activo"
16 }
```
- Response (Body):** 200 OK • 61 ms • 288 B. The response body is a JSON object:

```
1 {
2   "mensaje": "Alumno creado"
3 }
```

Método Put

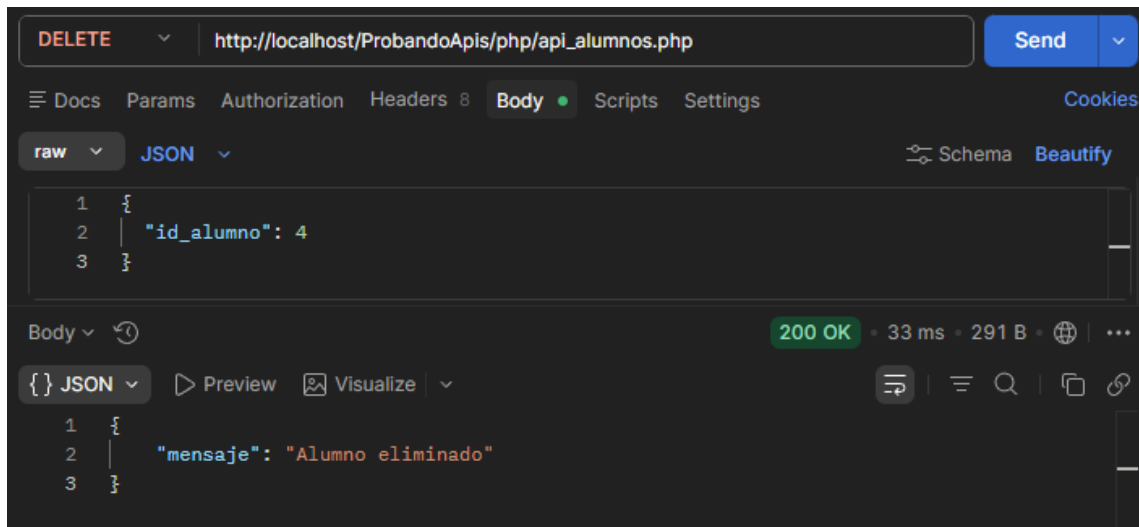
The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** `http://localhost/ProbandoApis/php/api_alumnos.php`
- Body (raw JSON):**

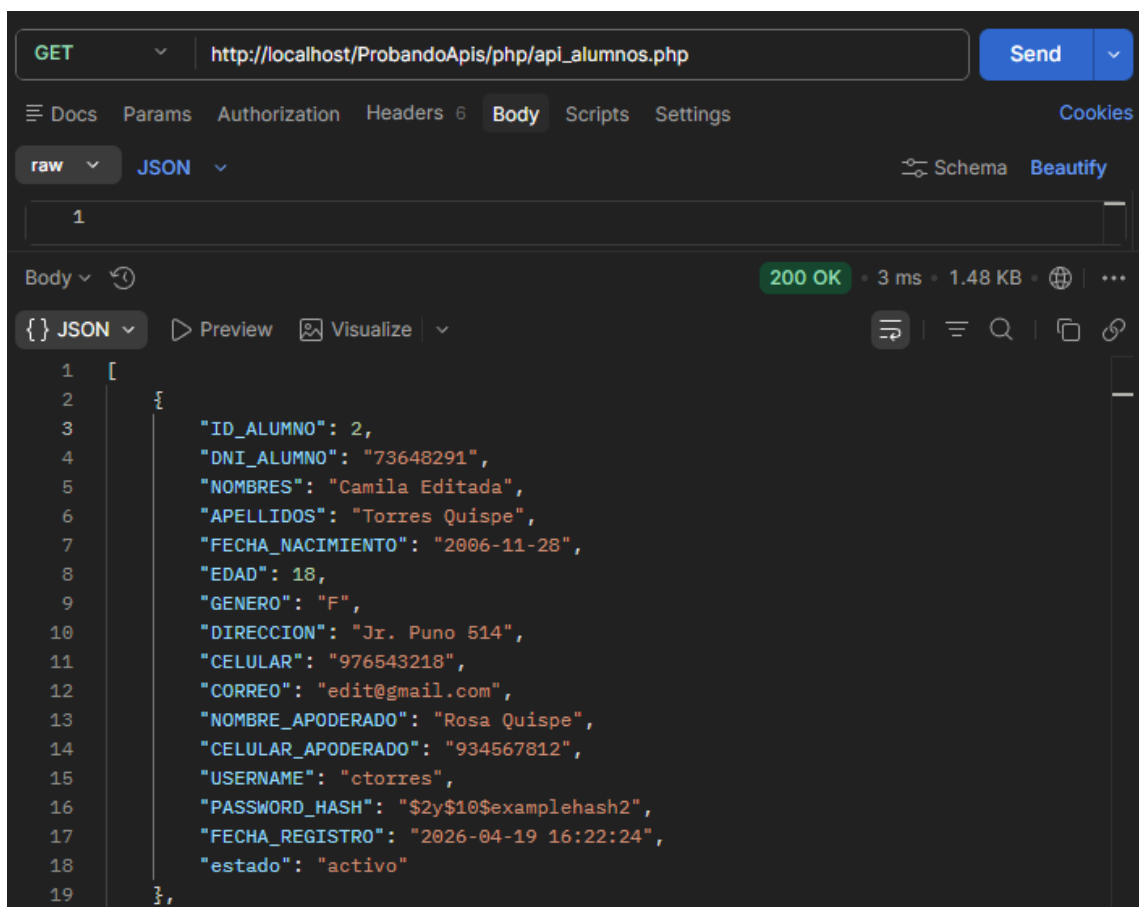
```
1 {
2   "id_alumno": 2,
3   "dni": "73648291",
4   "nombres": "Camila Editada",
5   "apellidos": "Torres Quispe",
6   "fecha_nac": "2006-11-28",
7   "edad": 18,
8   "genero": "F",
9   "direccion": "Jr. Puno 514",
10  "celular": "976543218",
11  "correo": "edit@gmail.com",
12  "apoderado": "Rosa Quispe",
13  "cel_apoderado": "934567812",
14  "username": "ctorres",
15  "estado": "activo"
16 }
```
- Response (Body):** 200 OK • 18 ms • 293 B. The response body is a JSON object:

```
1 {
2   "mensaje": "Alumno actualizado"
3 }
```

Método Delete



Comprobar actualización con get



Api_curso.php

```
1  <?php
2  header("Content-Type: application/json; charset=UTF-8");
3
4  // ===== CONEXIÓN =====
5  $host = "localhost";
6  $db = "MATRICULA";
7  $user = "root";
8  $pass = "";
9
10 try {
11     $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
12     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
13 } catch (PDOException $e) {
14     echo json_encode(["mensaje" => "Error de conexión"]);
15     exit;
16 }
17
18 // ===== VARIABLES =====
19 $metodo = $_SERVER['REQUEST_METHOD'];
20 $datos = json_decode(file_get_contents("php://input"));
21
22 switch ($metodo) {
23
24     // ===== GET =====
25     case 'GET':
26         if (isset($_GET['id'])) {
27             $stmt = $pdo->prepare("SELECT * FROM CURSO WHERE ID_CURSO = :id");
28             $stmt->execute(['id' => $_GET['id']]);
29             echo json_encode($stmt->fetch(PDO::FETCH_ASSOC));
30         } else {
31             $stmt = $pdo->query("SELECT * FROM CURSO");
32             echo json_encode($stmt->fetchAll(PDO::FETCH_ASSOC));
33         }
34         break;
35
36     // ===== POST =====
37     case 'POST':
38         $sql = "INSERT INTO CURSO (NOMBRE, DESCRIPCION, HORAS_SEMANA)
39             VALUES (:nombre, :descripcion, :horas)";
40         $stmt = $pdo->prepare($sql);
41
42         $stmt->execute([
43             'nombre' => $datos->nombre,
44             'descripcion' => $datos->descripcion,
45             'horas' => $datos->horas
46         ]);
47
48         echo json_encode(["mensaje" => "Curso creado"]);
49         break;
```



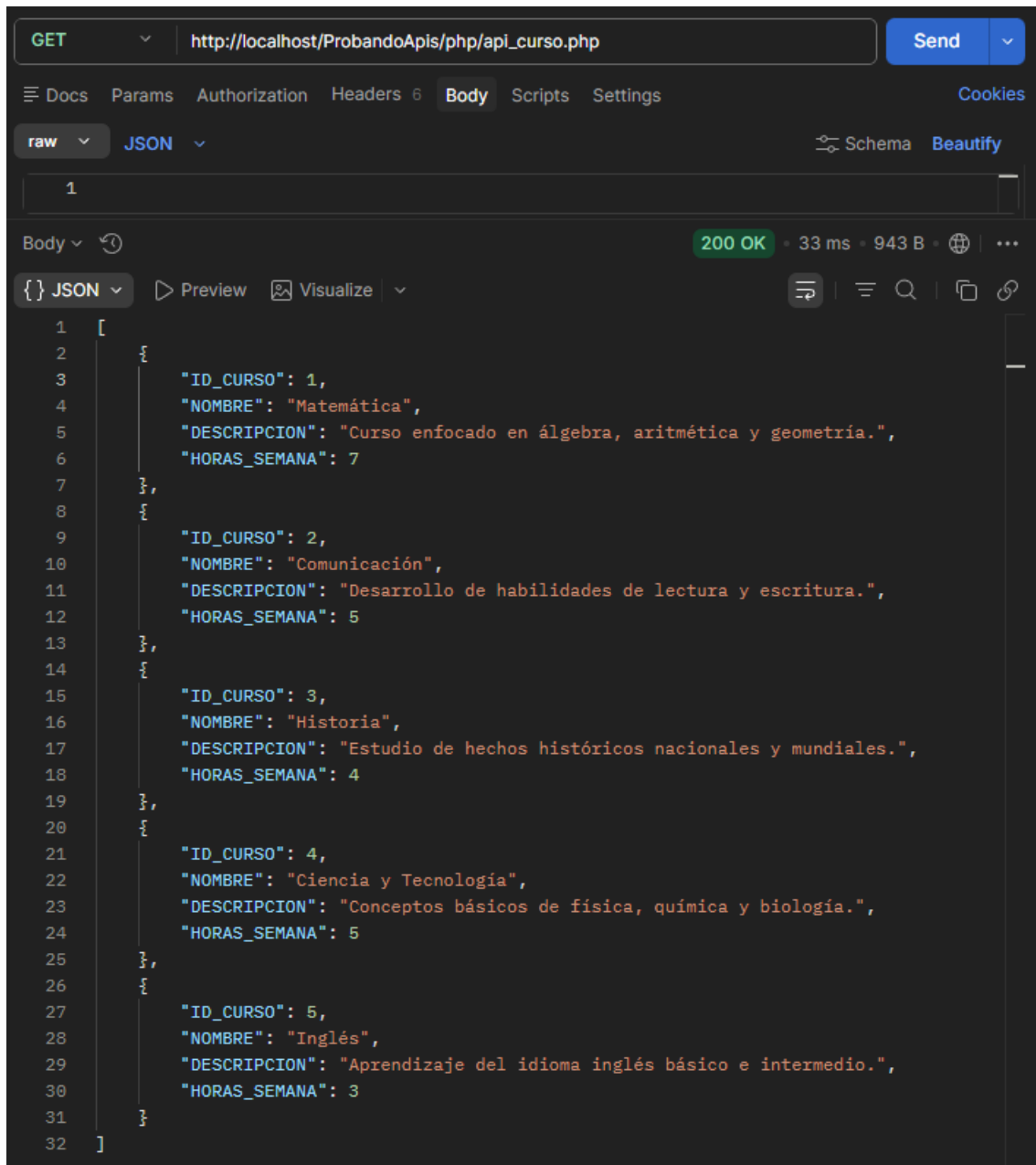
```

51 // ===== PUT =====
52 case 'PUT':
53     $sql = "UPDATE CURSO SET
54             NOMBRE=:nombre,
55             DESCRIPCION=:descripcion,
56             HORAS_SEMANA=:horas
57             WHERE ID_CURSO=:id";
58
59     $stmt = $pdo->prepare($sql);
60
61     $stmt->execute([
62         'nombre' => $datos->nombre,
63         'descripcion' => $datos->descripcion,
64         'horas' => $datos->horas,
65         'id' => $datos->id
66     ]);
67
68     echo json_encode(["mensaje" => "Curso actualizado"]);
69     break;
70
71 // ===== DELETE =====
72 case 'DELETE':
73     $stmt = $pdo->prepare("DELETE FROM CURSO WHERE ID_CURSO=:id");
74     $stmt->execute(['id' => $datos->id]);
75
76     echo json_encode(["mensaje" => "Curso eliminado"]);
77     break;
78
79 default:
80     http_response_code(405);
81     echo json_encode(["mensaje" => "Método no permitido"]);
82     break;
83 }

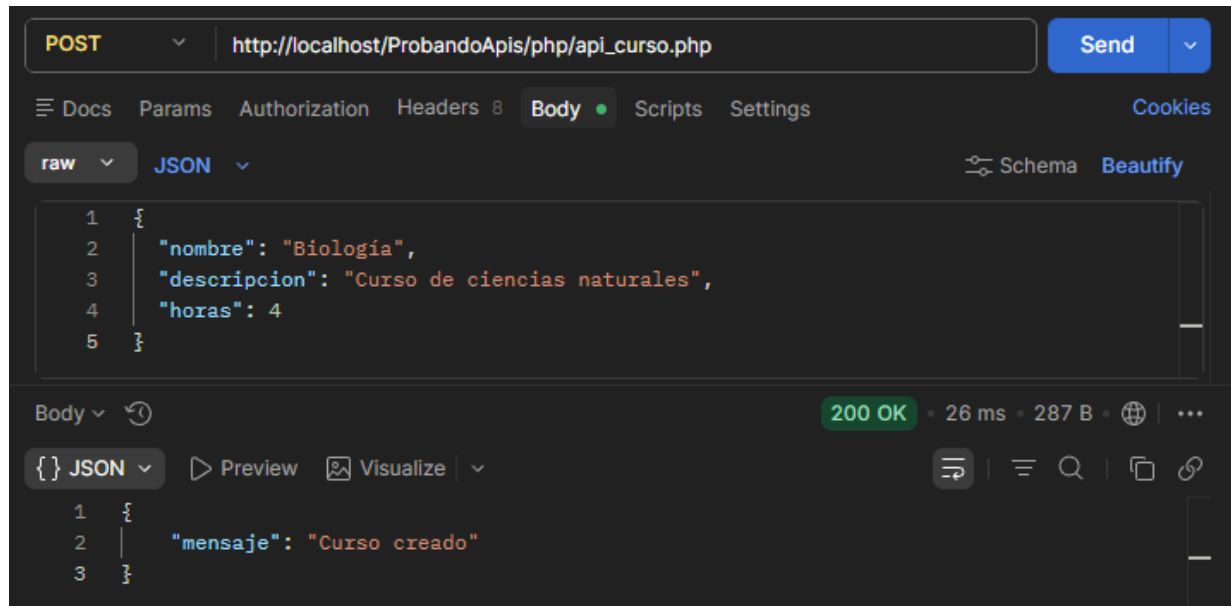
```

POSTMAN -- Api_curso.php

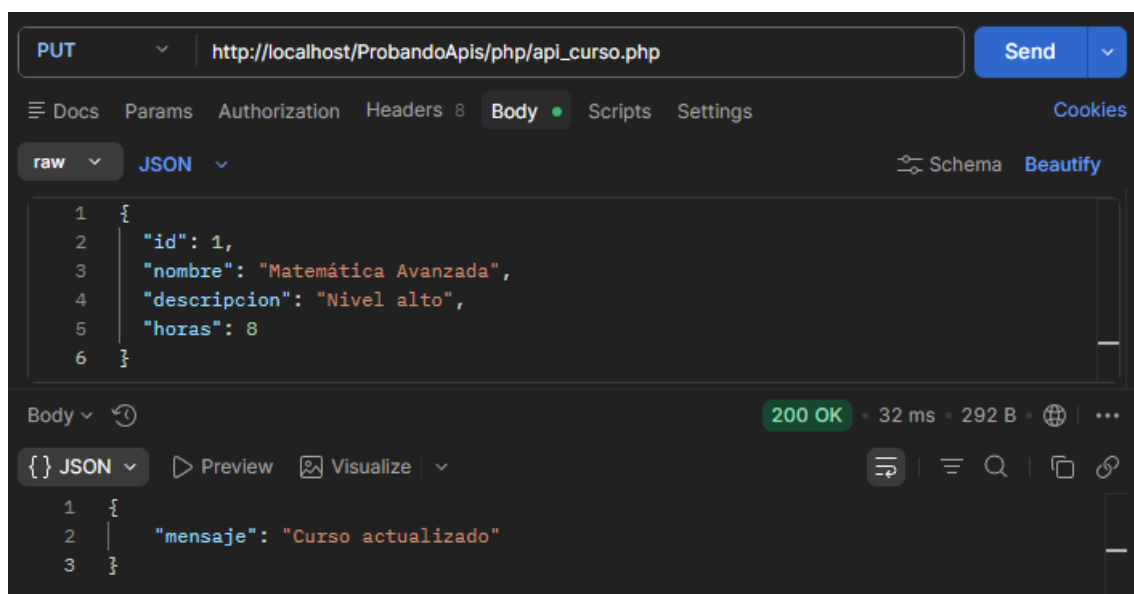
Método Get



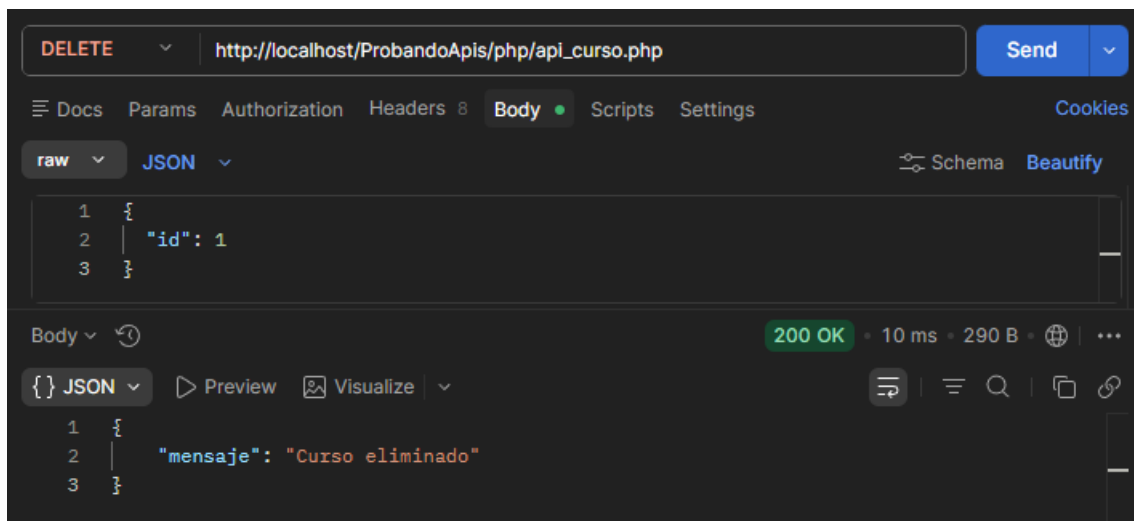
Método Post



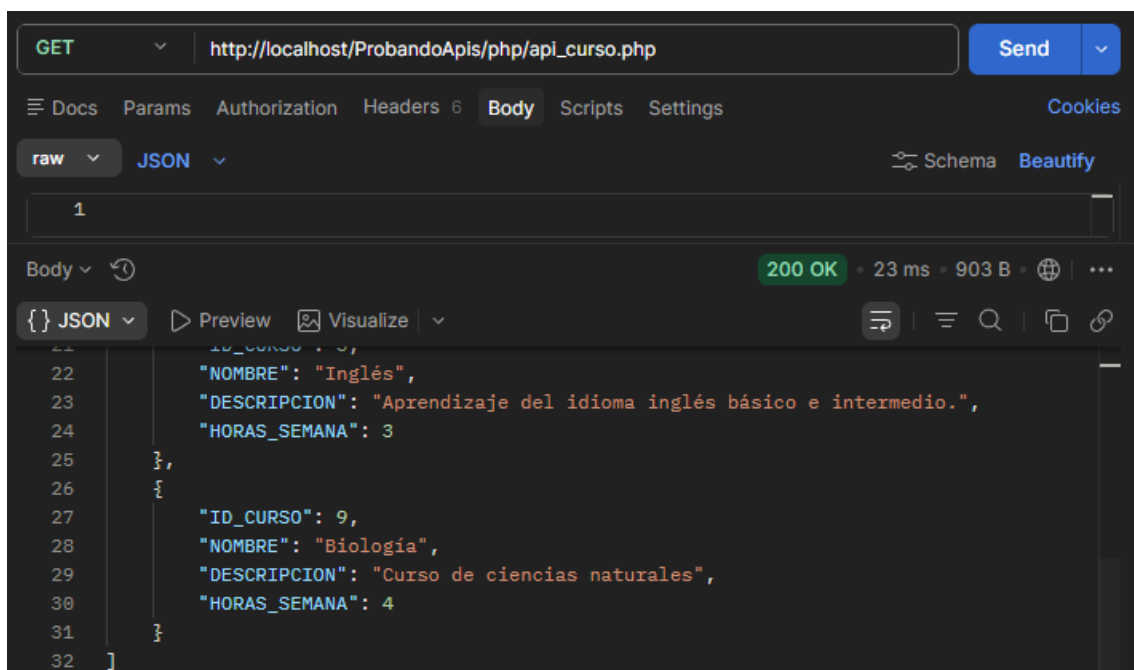
Método Put



Método Delete



Comprobar actualización con get



Api_aula.php

```
1  <?php
2  header("Content-Type: application/json; charset=UTF-8");
3
4  // ===== CONEXIÓN =====
5  $host = "localhost";
6  $db = "MATRICULA";
7  $user = "root";
8  $pass = "";
9
10 try {
11     $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
12     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
13 } catch (PDOException $e) {
14     echo json_encode(["mensaje" => "Error de conexión"]);
15     exit;
16 }
17
18 // ===== VARIABLES =====
19 $metodo = $_SERVER['REQUEST_METHOD'];
20 $datosJSON = json_decode(file_get_contents("php://input"));
21
22 // ===== SWITCH =====
23 switch ($metodo) {
24
25     // ===== GET =====
26     case 'GET':
27         try {
28
29             if (isset($_GET['id'])) {
30
31                 $sql = "SELECT * FROM AULA WHERE ID_AULA = :id";
32                 $stmt = $pdo->prepare($sql);
33                 $stmt->execute(['id' => $_GET['id']]);
34                 $dato = $stmt->fetch(PDO::FETCH_ASSOC);
35
36                 if ($dato) {
37                     echo json_encode($dato);
38                 } else {
39                     http_response_code(404);
40                     echo json_encode(["mensaje" => "Aula no encontrada"]);
41                 }
42
43             } else {
44                 $sql = "SELECT * FROM AULA";
45                 $stmt = $pdo->prepare($sql);
46                 $stmt->execute();
47
48                 echo json_encode($stmt->fetchAll(PDO::FETCH_ASSOC));
49             }
50         } catch (Exception $e) {
51             http_response_code(500);
52             echo json_encode([
53                 "mensaje" => "Error al obtener datos",
54                 "error" => $e->getMessage()
55             ]);
56         }
57         break;
58
59     // ===== POST =====
60     case 'POST':
61         if (!empty($datosJSON->nivel) && !empty($datosJSON->grado) && !empty($datosJSON->seccion)) {
```

```

64         try {
65             $sql = "INSERT INTO AULA
66                 (NIVEL, GRADO, SECCION, VACANTES_TOTALES, VACANTES_DISPONIBLES)
67                 VALUES (:nivel, :grado, :seccion, :totales, :disponibles)";
68
69             $stmt = $pdo->prepare($sql);
70
71             $stmt->execute([
72                 'nivel' => $datosJSON->nivel,
73                 'grado' => $datosJSON->grado,
74                 'seccion' => $datosJSON->seccion,
75                 'totales' => $datosJSON->vacantes_totales ?? 0,
76                 'disponibles' => $datosJSON->vacantes_disponibles ?? 0
77             ]);
78
79             echo json_encode(["mensaje" => "Aula creada correctamente"]);
80
81         } catch (Exception $e) {
82             http_response_code(500);
83             echo json_encode([
84                 "mensaje" => "Error al crear aula",
85                 "error" => $e->getMessage()
86             ]);
87         }
88     } else {
89         http_response_code(400);
90         echo json_encode(["mensaje" => "Datos incompletos"]);
91     }
92     break;
93
94     // ===== PUT =====
95     case 'PUT':
96         if (!empty($datosJSON->id_aula)) {
97
98             try {
99                 $sql = "UPDATE AULA
100                     SET NIVEL=:nivel, GRADO=:grado, SECCION=:seccion,
101                     VACANTES_TOTALES=:totales, VACANTES_DISPONIBLES=:disponibles
102                     WHERE ID_AULA=:id";
103
104                 $stmt = $pdo->prepare($sql);
105
106                 $stmt->execute([
107                     'nivel' => $datosJSON->nivel,
108                     'grado' => $datosJSON->grado,
109                     'seccion' => $datosJSON->seccion,
110                     'totales' => $datosJSON->vacantes_totales ?? 0,
111                     'disponibles' => $datosJSON->vacantes_disponibles ?? 0,
112                     'id' => $datosJSON->id_aula
113                 ]);
114
115                 echo json_encode(["mensaje" => "Aula actualizada correctamente"]);
116
117             } catch (Exception $e) {
118                 http_response_code(500);
119                 echo json_encode([
120                     "mensaje" => "Error al actualizar aula",
121                     "error" => $e->getMessage()
122                 ]);
123             }
124
125         } else {
126             http_response_code(400);
127             echo json_encode(["mensaje" => "Falta ID"]);
128         }
129         break;

```

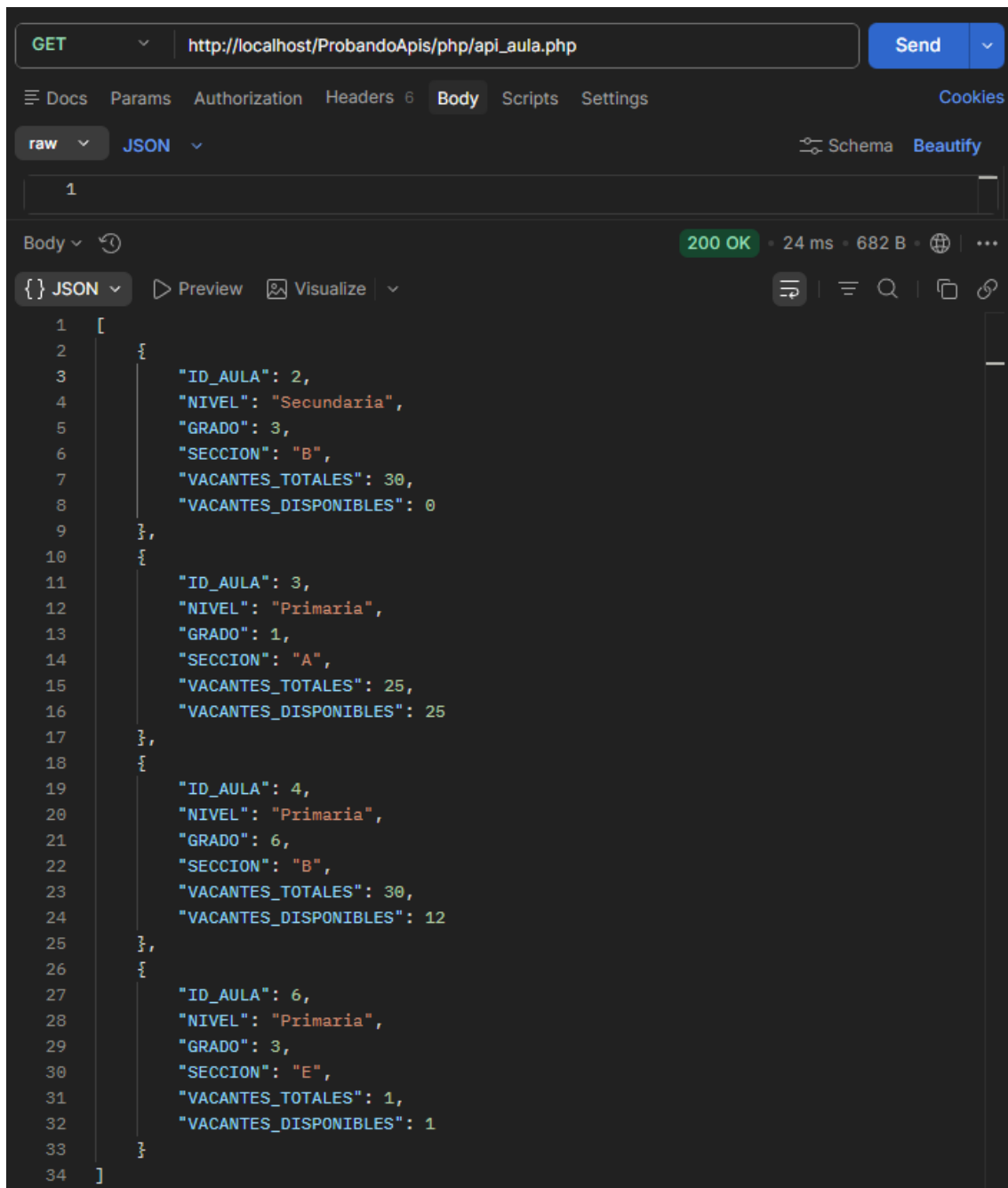
```

132 // ===== DELETE =====
133 case 'DELETE':
134     if (!empty($datosJSON->id_aula)) {
135
136         try {
137             $pdo->beginTransaction();
138
139             $stmt = $pdo->prepare("DELETE FROM RESERVA WHERE ID_AULA = :id");
140             $stmt->execute(['id' => $datosJSON->id_aula]);
141
142             $stmt = $pdo->prepare("DELETE FROM AULA WHERE ID_AULA = :id");
143             $stmt->execute(['id' => $datosJSON->id_aula]);
144
145             $pdo->commit();
146
147             echo json_encode(["mensaje" => "Aula eliminada correctamente"]);
148
149         } catch (Exception $e) {
150             $pdo->rollBack();
151             http_response_code(500);
152             echo json_encode([
153                 "mensaje" => "Error al eliminar aula",
154                 "error" => $e->getMessage()
155             ]);
156         }
157
158     } else {
159         http_response_code(400);
160         echo json_encode(["mensaje" => "Falta ID"]);
161     }
162     break;
163
164 // ===== DEFAULT =====
165 default:
166     http_response_code(405);
167     echo json_encode(["mensaje" => "Método no permitido"]);
168     break;
169 }
170 ?>

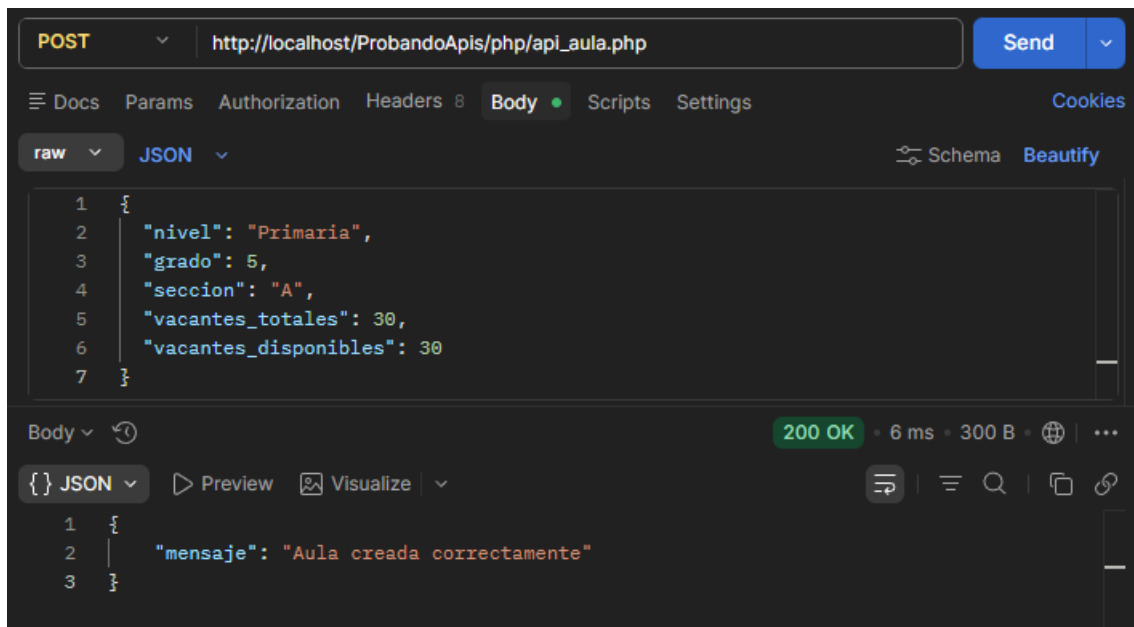
```

POSTMAN -- Api_aula.php

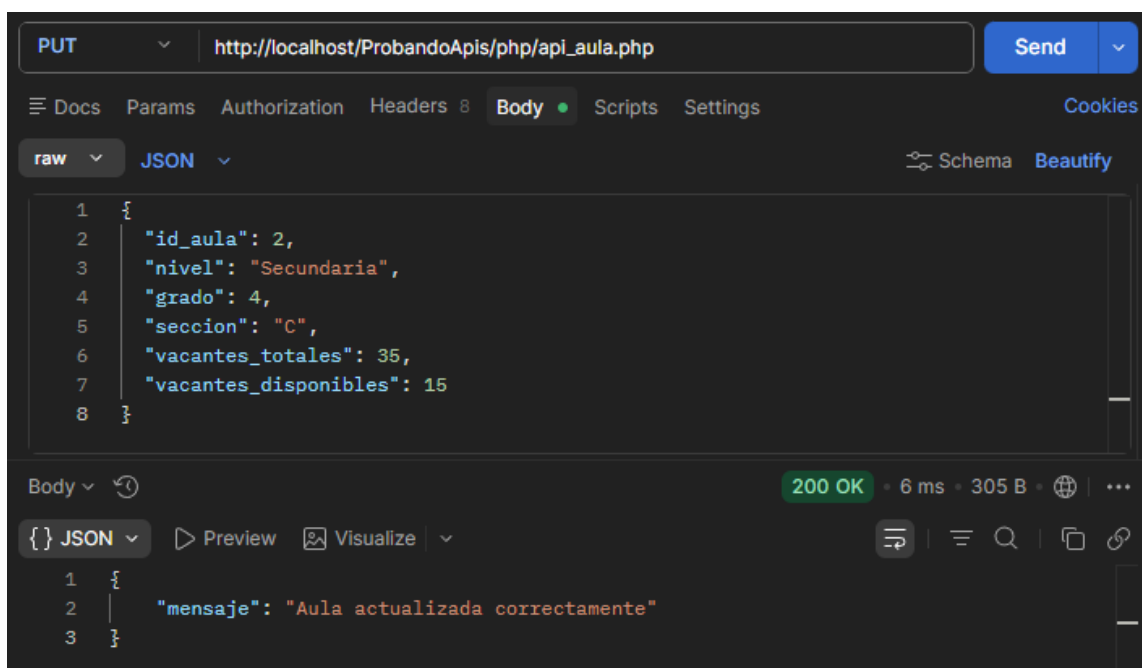
Método Get



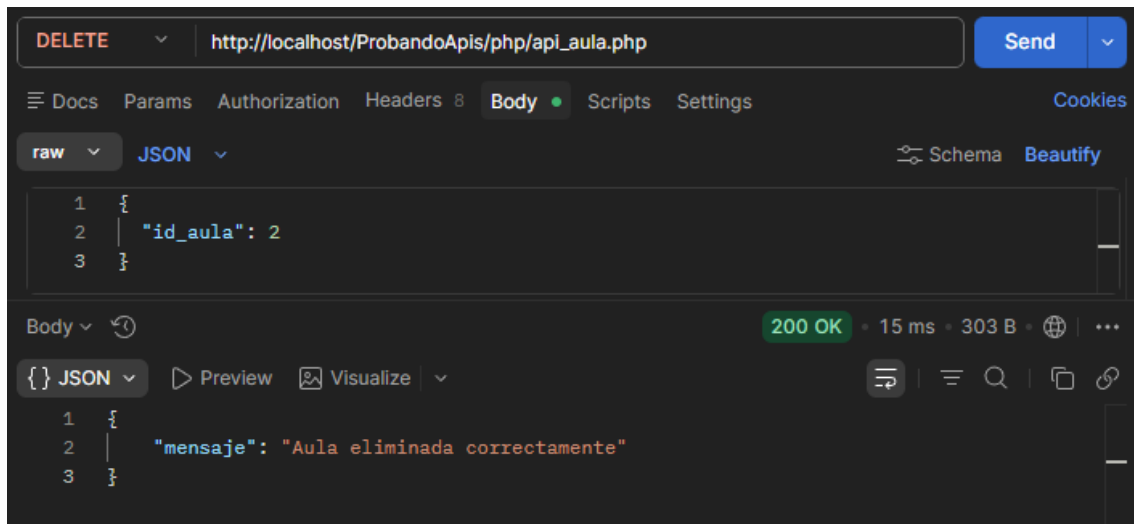
Método Post



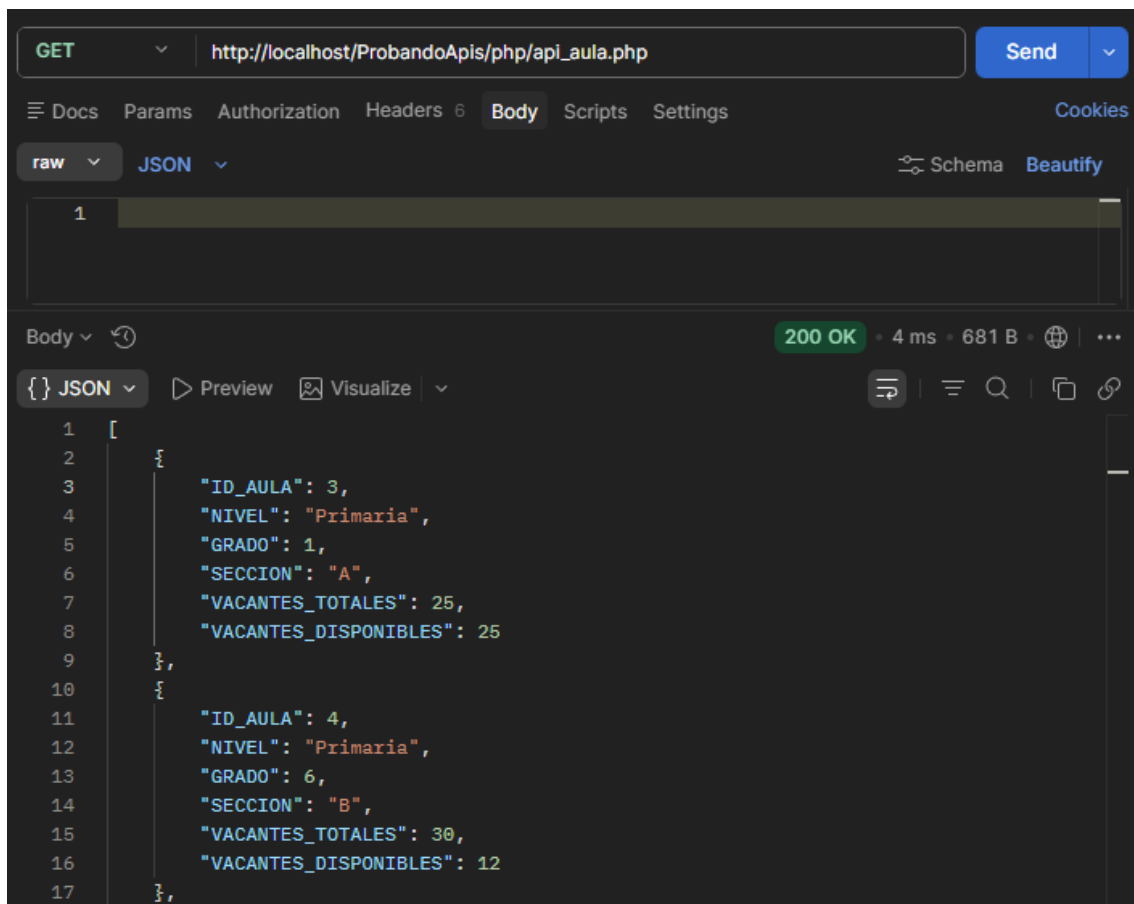
Método PUT



Método Delete



Comprobar actualización con get



Api_matricula.php

```
1  <?php
2  header('Content-Type: application/json');
3
4  // ===== CONEXIÓN =====
5  $host = "localhost";
6  $db = "MATRICULA";
7  $user = "root";
8  $pass = "";
9
10 try {
11     $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
12     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
13 } catch (PDOException $e) {
14     echo json_encode(["mensaje" => "Error de conexión"]);
15     exit;
16 }
17
18 $metodo = $_SERVER['REQUEST_METHOD'];
19 $datos = json_decode(file_get_contents("php://input"));
20
21 switch ($metodo) {
22
23     // ===== GET =====
24     case 'GET':
25
26         $sql = "SELECT
27                 a.ID_ALUMNO,
28                 CONCAT(a.NOMBRES, ' ', a.APELLIDOS) AS ESTUDIANTE,
29                 a.DNI_ALUMNO,
30                 a.CELULAR,
31                 m.ESTADO_MATRICULA,
32                 CASE
33                     WHEN m.ID_MATRICULA IS NULL THEN 'Sin matricula'
34                     ELSE CONCAT(au.NIVEL, ' ', au.GRADO, 'º ', au.SECCION)
35                 END AS MATRICULA_ACTUAL,
36                 a.ESTADO
37             FROM ALUMNO a
38             LEFT JOIN MATRICULA m
39                 ON a.ID_ALUMNO = m.ID_ALUMNO
40                 AND m.ANIO_ESCOLAR = YEAR(CURDATE())
41             LEFT JOIN AULA au
42                 ON m.ID_AULA = au.ID_AULA";
43
44         $stmt = $pdo->prepare($sql);
45         $stmt->execute();
46
47         echo json_encode($stmt->fetchAll(PDO::FETCH_ASSOC));
48         break;
49
50     // ===== POST =====
51     case 'POST':
52
53         $id_alumno = $datos->id_alumno ?? '';
54         $nivel = $datos->nivel ?? '';
55         $grado = $datos->grado ?? '';
56
57         if (!$id_alumno || !$nivel || !$grado) {
58             echo json_encode(["mensaje" => "Faltan datos"]);
59             exit;
60         }
61     }
```

```

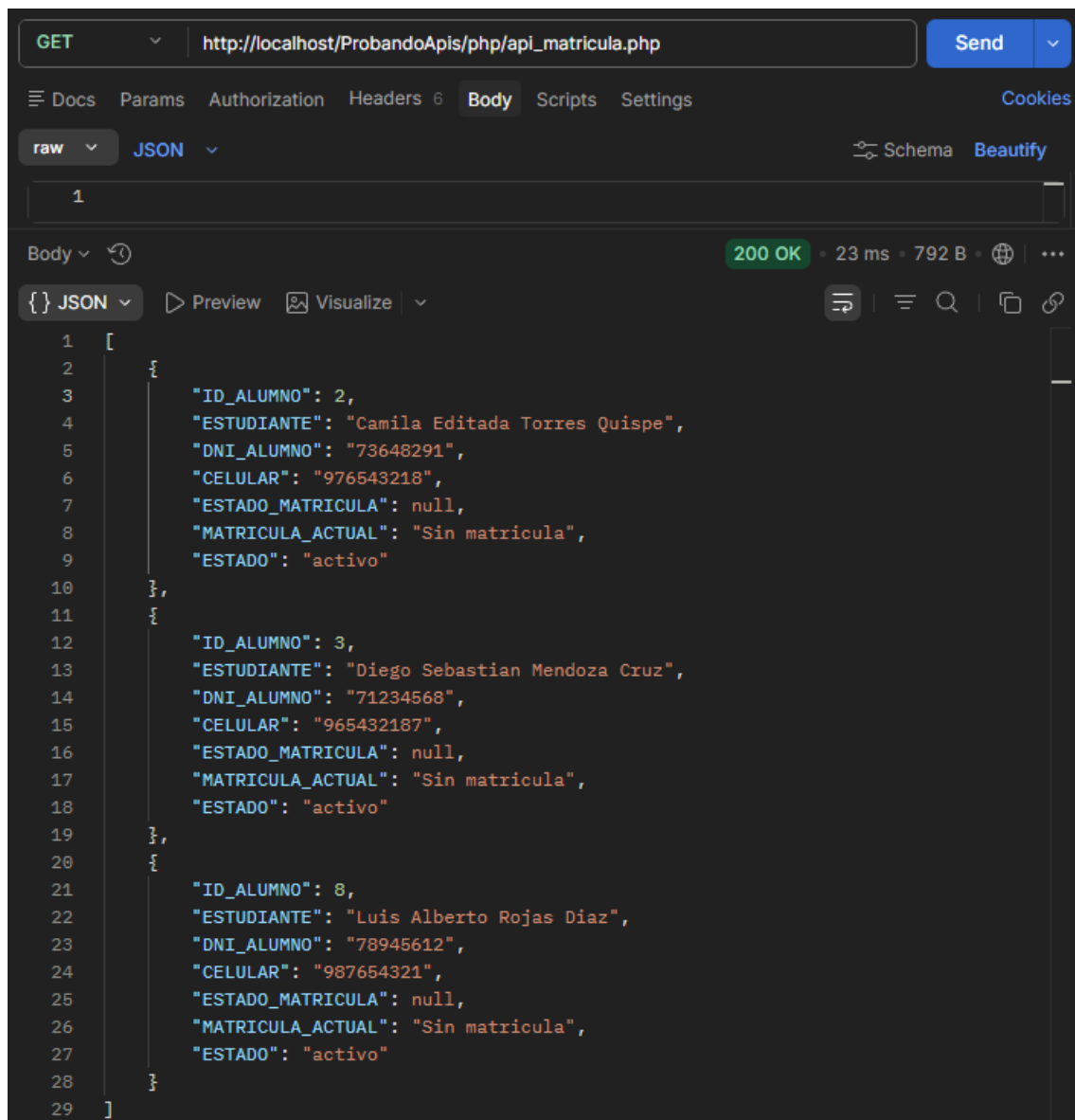
62     $sql = "SELECT ID_MATRICULA
63             FROM MATRICULA
64             WHERE ID_ALUMNO = ? AND ANIO_ESCOLAR = YEAR(CURDATE())";
65     $stmt = $pdo->prepare($sql);
66     $stmt->execute([$id_alumno]);
67
68     if ($stmt->fetch()) {
69         echo json_encode(["mensaje" => "Ya matriculado"]);
70         exit;
71     }
72
73     $sql = "SELECT ID_AULA FROM AULA
74             WHERE LOWER(NIVEL)=LOWER(?) AND GRADO=? LIMIT 1";
75     $stmt = $pdo->prepare($sql);
76     $stmt->execute([$nivel, $grado]);
77     $aula = $stmt->fetch();
78
79     if (!$aula) {
80         echo json_encode(["mensaje" => "Aula no encontrada"]);
81         exit;
82     }
83
84     $id_aula = $aula['ID_AULA'];
85
86     try {
87         $pdo->beginTransaction();
88
89         $sql = "INSERT INTO MATRICULA
90                 (ID_ALUMNO, ID_AULA, FECHA_MATRICULA, ANIO_ESCOLAR, ESTADO_MATRICULA)
91                 VALUES (?, ?, NOW(), YEAR(CURDATE()), 'ACTIVO')";
92         $stmt = $pdo->prepare($sql);
93         $stmt->execute([$id_alumno, $id_aula]);
94
95         $codigo_pago = "PAGO" . str_pad($id_alumno, 3, "0", STR_PAD_LEFT);
96
97         $sql = "INSERT INTO RESERVA
98                 (ID_ALUMNO, ID_AULA, CODIGO_PAGO, FECHA_RESERVA, ESTADO_PAGOO)
99                 VALUES (?, ?, ?, NOW(), 'PENDIENTE')";
100        $stmt = $pdo->prepare($sql);
101        $stmt->execute([$id_alumno, $id_aula, $codigo_pago]);
102
103        $pdo->commit();
104
105        echo json_encode(["mensaje" => "Matrícula registrada"]);
106    } catch (Exception $e) {
107        $pdo->rollBack();
108        echo json_encode([
109            "mensaje" => "Error",
110            "error" => $e->getMessage()
111        ]);
112    }
113    break;
114
115    // ===== DELETE =====
116    case 'DELETE':
117
118        $id_alumno = $datos->id_alumno ?? '';
119
120        if (!$id_alumno) {
121            echo json_encode(["mensaje" => "ID inválido"]);
122            exit;
123        }
124

```

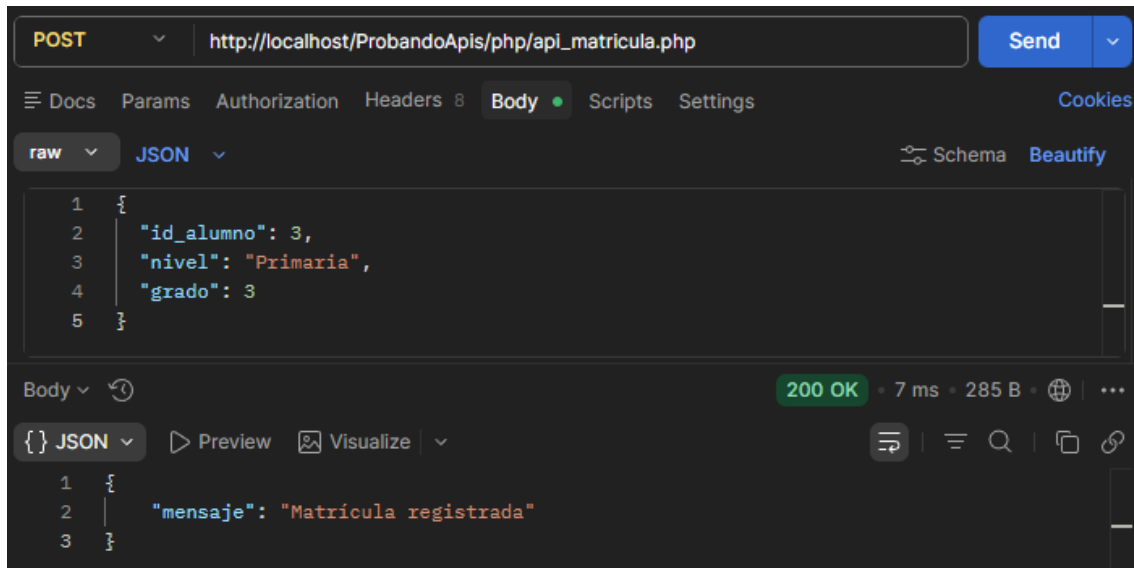
```
126     try {
127         $pdo->beginTransaction();
128
129         $stmt = $pdo->prepare("DELETE FROM MATRICULA
130                                WHERE ID_ALUMNO=? AND ANIO_ESCOLAR=YEAR(CURDATE())");
131         $stmt->execute([$id_alumno]);
132
133         $stmt = $pdo->prepare("DELETE FROM RESERVA WHERE ID_ALUMNO=?");
134         $stmt->execute([$id_alumno]);
135
136         $pdo->commit();
137
138         echo json_encode(["mensaje" => "Matrícula cancelada"]);
139
140     } catch (Exception $e) {
141         $pdo->rollBack();
142         echo json_encode(["mensaje" => "Error al cancelar"]);
143     }
144     break;
145 }
146 ?>
```

POSTMAN -- Api_matricula.php

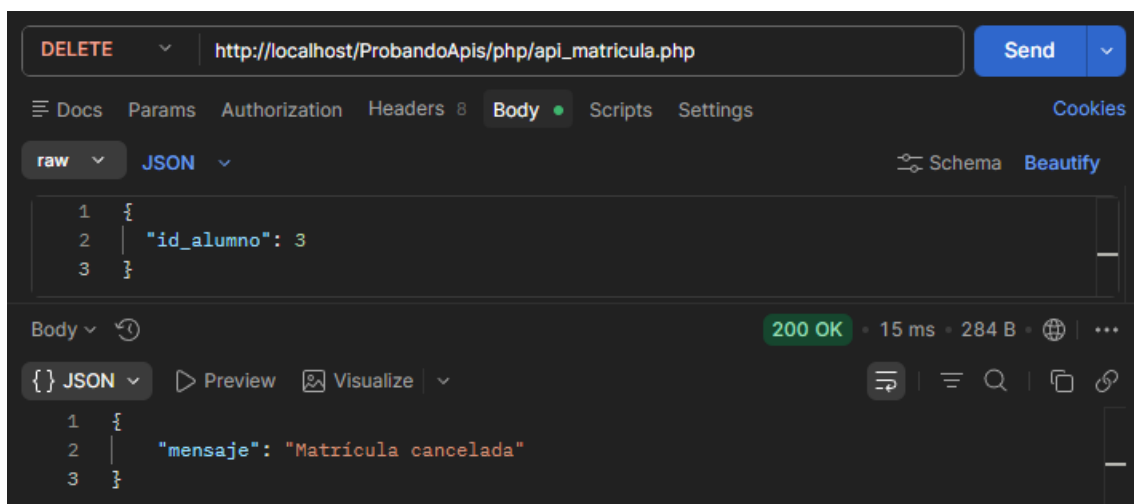
Método Get



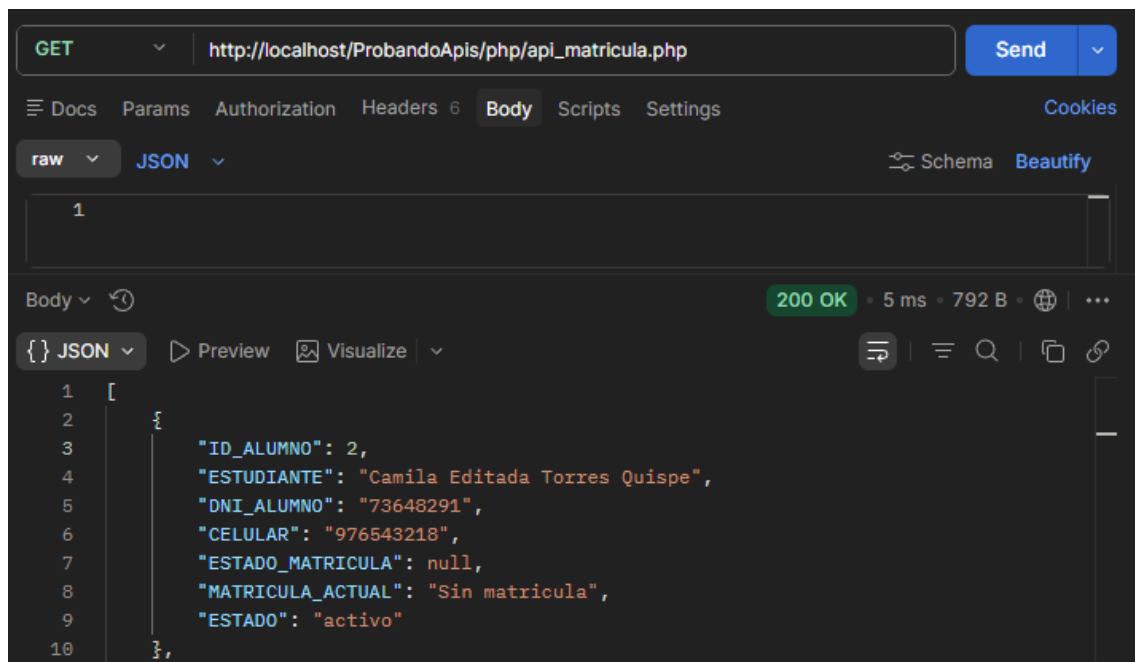
Método Post



Método Delete



Comprobar actualización con get



Api_pagos.php

```
1  <?php
2  header("Content-Type: application/json; charset=UTF-8");
3
4  // ===== CONEXIÓN =====
5  $host = "localhost";
6  $db = "MATRICULA";
7  $user = "root";
8  $pass = "";
9
10 try {
11     $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
12     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
13 } catch (PDOException $e) {
14     echo json_encode(["mensaje" => "Error de conexión"]);
15     exit;
16 }
17
18 // ===== VARIABLES =====
19 $metodo = $_SERVER['REQUEST_METHOD'];
20 $datos = json_decode(file_get_contents("php://input"));
21
22 switch ($metodo) {
23
24     // ===== GET =====
25     case 'GET':
26         try {
27
28             $sql = "SELECT
29                     r.ID_RESERVA,
30                     r.CODIGO_PAGO,
31                     r.FECHA_RESERVA,
32                     r.ESTADO_PAGOO,
33                     CONCAT(a.NOMBRES, ' ', a.APELLIDOS) AS ALUMNO,
34                     au.NIVEL,
35                     au.GRADO,
36                     au.SECCION
37                 FROM RESERVA r
38                 JOIN ALUMNO a ON r.ID_ALUMNO = a.ID_ALUMNO
39                 JOIN AULA au ON r.ID_AULA = au.ID_AULA";
40
41             $stmt = $pdo->prepare($sql);
42             $stmt->execute();
43
44             echo json_encode($stmt->fetchAll(PDO::FETCH_ASSOC));
45
46         } catch (Exception $e) {
47             echo json_encode([
48                 "mensaje" => "Error al obtener pagos",
49                 "error" => $e->getMessage()
50             ]);
51         }
52     }
53 }
```

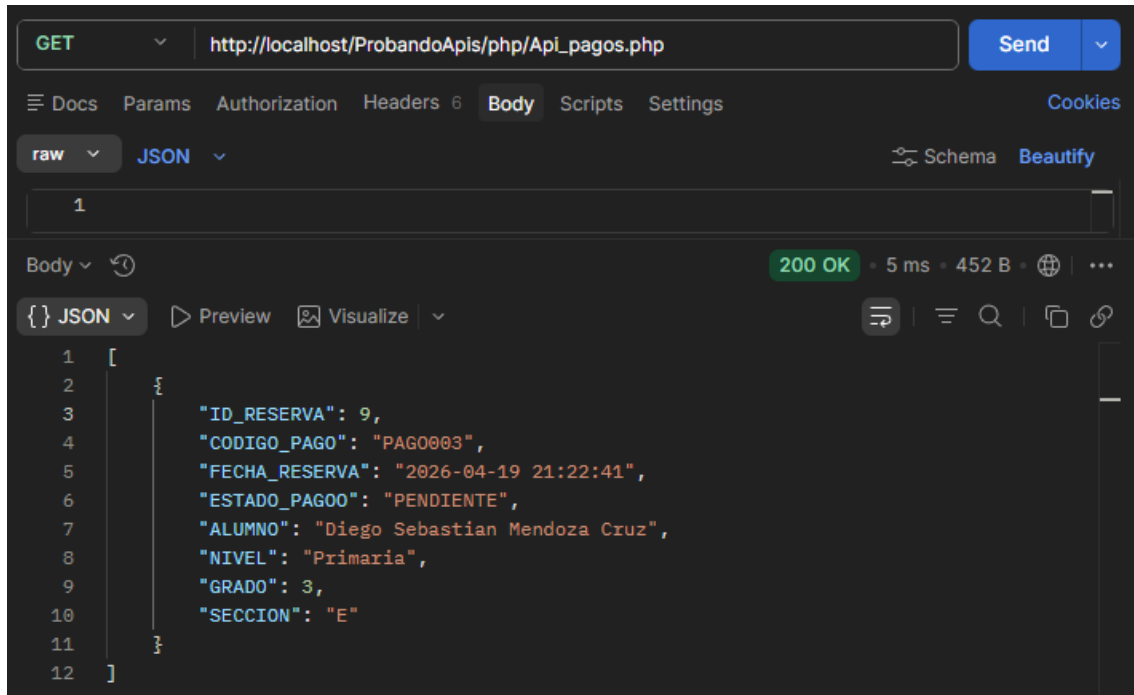
```

51     }
52     break;
53
54
55     // ===== PUT (MARCAR PAGADO) =====
56     case 'PUT':
57         if (!empty($datos->id_reserva)) {
58
59             try {
60                 $sql = "UPDATE RESERVA
61                     SET ESTADO_PAGOO = 'PAGADO'
62                     WHERE ID_RESERVA = :id";
63
64                 $stmt = $pdo->prepare($sql);
65                 $stmt->execute(['id' => $datos->id_reserva]);
66
67                 echo json_encode(["mensaje" => "Pago actualizado"]);
68
69             } catch (Exception $e) {
70                 echo json_encode([
71                     "mensaje" => "Error al actualizar pago",
72                     "error" => $e->getMessage()
73                 ]);
74             }
75
76         } else {
77             echo json_encode(["mensaje" => "Falta ID"]);
78         }
79         break;
80
81
82     // ===== DELETE =====
83     case 'DELETE':
84         if (!empty($datos->id_reserva)) {
85
86             try {
87                 $stmt = $pdo->prepare("DELETE FROM RESERVA WHERE ID_RESERVA = :id");
88                 $stmt->execute(['id' => $datos->id_reserva]);
89
90                 echo json_encode(["mensaje" => "Pago eliminado"]);
91
92             } catch (Exception $e) {
93                 echo json_encode([
94                     "mensaje" => "Error al eliminar",
95                     "error" => $e->getMessage()
96                 ]);
97             }
98
99         } else {
100             echo json_encode(["mensaje" => "Falta ID"]);
101         }
102         break;
103
104
105     default:
106         http_response_code(405);
107         echo json_encode(["mensaje" => "Método no permitido"]);
108         break;
109 }
110 >>

```

POSTMAN -- Api_pagos.php

Método Get



Api_usuario.php

```
1  <?php
2  header("Content-Type: application/json; charset=UTF-8");
3
4  // ===== CONEXIÓN =====
5  $host = "localhost";
6  $db = "MATRICULA";
7  $user = "root";
8  $pass = "";
9
10 try {
11     $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
12     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
13 } catch (PDOException $e) {
14     echo json_encode(["mensaje" => "Error de conexión"]);
15     exit;
16 }
17
18 // ===== VARIABLES =====
19 $metodo = $_SERVER['REQUEST_METHOD'];
20 $datos = json_decode(file_get_contents("php://input"));
21
22 switch ($metodo) {
23
24     // ===== GET =====
25     case 'GET':
26         try {
27             $stmt = $pdo->prepare("SELECT ID, USERNAME, ESTADO FROM USUARIO");
28             $stmt->execute();
29
30             echo json_encode($stmt->fetchAll(PDO::FETCH_ASSOC));
31         } catch (Exception $e) {
32             echo json_encode(["mensaje" => "Error al obtener usuarios"]);
33         }
34         break;
35
36     // ===== POST =====
37     case 'POST':
38         if (!empty($datos->username) && !empty($datos->password)) {
39
40             try {
41                 $passwordHash = password_hash($datos->password, PASSWORD_DEFAULT);
42
43                 $sql = "INSERT INTO USUARIO (USERNAME, PASSWORD_HASH, ESTADO)
44                     VALUES (:username, :password, :estado)";
45
46                 $stmt = $pdo->prepare($sql);
47
48                 $stmt->execute([
49                     'username' => $datos->username,
50                     'password' => $passwordHash,
51                     'estado' => $datos->estado ?? 1
52                 ]);
53
54                 echo json_encode(["mensaje" => "Usuario creado"]);
55             } catch (Exception $e) {
56                 echo json_encode([
57                     "mensaje" => "Error al crear usuario",
58                     "error" => $e->getMessage()
59                 ]);
60             }
61         }
```

```

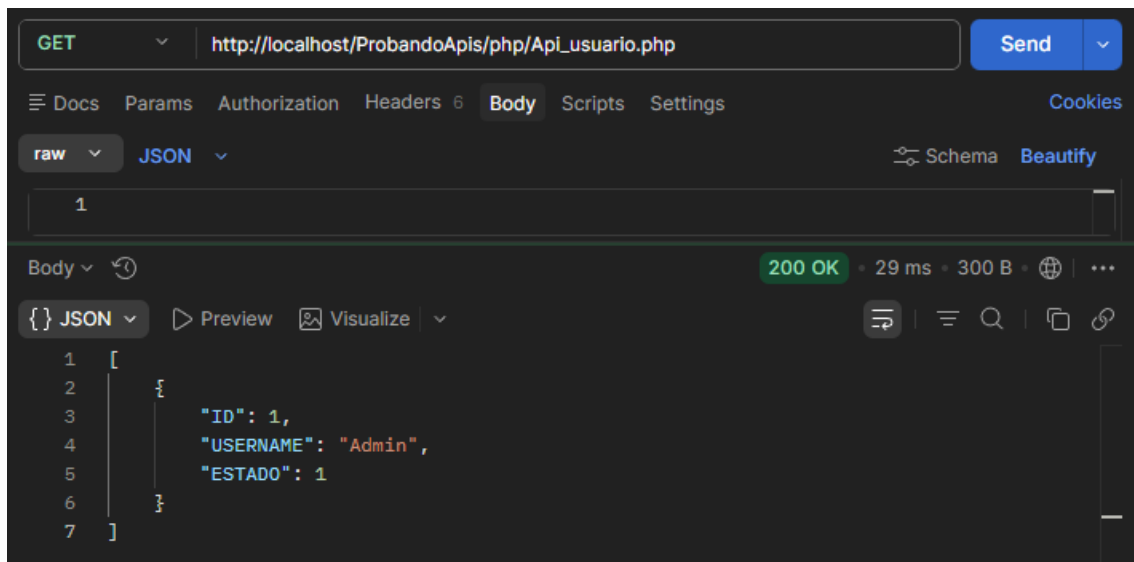
62     }
63
64     } else {
65         echo json_encode(["mensaje" => "Faltan datos"]);
66     }
67     break;
68
69
70     // ===== PUT =====
71     case 'PUT':
72         if (!empty($datos->id)) {
73
74             try {
75
76                 if (!empty($datos->password)) {
77
78                     $passwordHash = password_hash($datos->password, PASSWORD_DEFAULT);
79
80                     $sql = "UPDATE USUARIO
81                             SET USERNAME=:username, PASSWORD_HASH=:password, ESTADO=:estado
82                             WHERE ID=:id";
83
84                     $stmt = $pdo->prepare($sql);
85
86                     $stmt->execute([
87                         'username' => $datos->username,
88                         'password' => $passwordHash,
89                         'estado' => $datos->estado,
90                         'id' => $datos->id
91                     ]);
92
93                 } else {
94
95                     $sql = "UPDATE USUARIO
96                             SET USERNAME=:username, ESTADO=:estado
97                             WHERE ID=:id";
98
99                     $stmt = $pdo->prepare($sql);
100
101                     $stmt->execute([
102                         'username' => $datos->username,
103                         'estado' => $datos->estado,
104                         'id' => $datos->id
105                     ]);
106                 }
107
108                 echo json_encode(["mensaje" => "Usuario actualizado"]);
109
110             } catch (Exception $e) {
111                 echo json_encode([
112                     "mensaje" => "Error al actualizar",
113                     "error" => $e->getMessage()
114                 ]);
115             }
116
117         } else {
118             echo json_encode(["mensaje" => "Falta ID"]);
119         }
120         break;
121

```

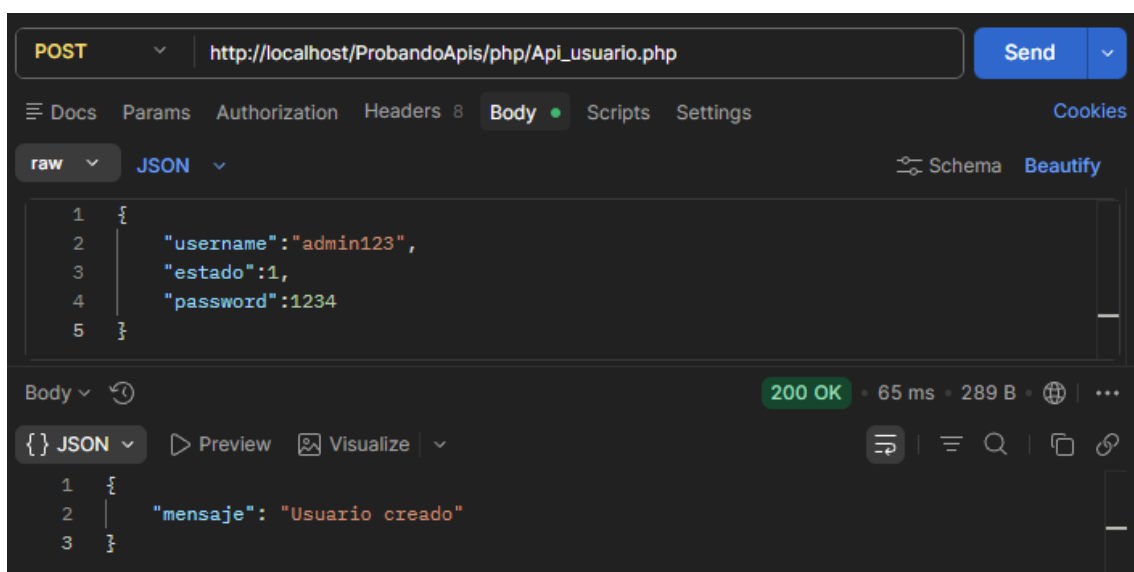
```
146     default:
147         http_response_code(405);
148         echo json_encode(["mensaje" => "Método no permitido"]);
149         break;
150     }
151     ?>
```

POSTMAN -- Api_usuario.php

Método Get



Método Post



Método PUT

PUT

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "id":1,
3   "username":"admin1",
4   "estado":1
5 }
```

Body 200 OK • 10 ms • 294 B •

Preview Visualize

```
1 {
2   "mensaje": "Usuario actualizado"
3 }
```

Método DELETE

DELETE

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "id": 3
3 }
```

Body 200 OK • 9 ms • 292 B •

Preview Visualize

```
1 {
2   "mensaje": "Usuario eliminado"
3 }
```